1. Resolución del Problema de los Filósofos Comensales usando Semáforos

Curso: Sistemas Operativos

Docente: Fabio Andres Guzman Figueroa

Trabajo 1: Sincronización con Semáforos

Estudiante: Santiago Hernández Morantes – Simón Bedoya Urrea

Fecha: 25 de febrero de 2026

1.1. Introducción

En sistemas concurrentes, múltiples procesos o hilos pueden competir por recursos compartidos. Cuando no existe un mecanismo de sincronización adecuado, pueden presentarse problemas como:

- Condiciones de carrera
- Interbloqueo (*deadlock*)
- Inanición (*starvation*)

Los semáforos, propuestos por Edsger Dijkstra, son una herramienta fundamental para controlar el acceso a recursos compartidos y evitar estos problemas.

En este trabajo se implementa el clásico problema de los Filósofos Comensales utilizando semáforos POSIX en C.

1.2. Descripción del Problema

El problema plantea:

- N filósofos sentados alrededor de una mesa circular.
- Entre cada par de filósofos hay un cubierto.
- Para comer, un filósofo necesita dos cubiertos: el de su izquierda y el de su derecha.

Cada filósofo alterna entre:

- Pensar
- Tener hambre
- Comer

El reto es garantizar que:

- No haya interbloqueo (*deadlock*).
- No haya uso simultáneo de un mismo cubierto.
- El sistema sea concurrente (no estrictamente secuencial).

1.3. Condiciones del Deadlock

Para que ocurra un deadlock deben cumplirse cuatro condiciones:

1. **Exclusión mutua:** los cubiertos no pueden compartirse.
2. **Hold and wait:** un filósofo puede tener un cubierto y esperar otro.
3. **No apropiación:** no se puede forzar a soltar un cubierto.
4. **Espera circular:** cada filósofo espera el recurso del siguiente.

Nuestra solución rompe la condición de **espera circular** mediante una verificación controlada del estado de los vecinos antes de habilitar el estado de comida.

1.4. Estrategia de Solución

Se implementó la solución clásica de Dijkstra basada en:

- Un semáforo global `mutex` para proteger la sección crítica.
- Un semáforo privado `s[i]` por cada filósofo.
- Un arreglo `state[i]` con estados: `THINKING`, `HUNGRY`, `EATING`.

Idea principal: un filósofo solo puede comer si está hambriento y sus dos vecinos no están comiendo. Si no cumple estas condiciones, queda bloqueado en su semáforo privado hasta que pueda continuar.

Esto permite:

- Evitar interbloqueo
- Maximizar concurrencia (no vecinos pueden comer en paralelo)
- Asegurar exclusión mutua sobre los cubiertos

1.5. Esquema del Algoritmo

Estados: `THINKING` → `HUNGRY` → `EATING` → `THINKING`

Funciones clave:

- `take_forks(i)`
- `put_forks(i)`
- `test(i)`

1.6. Pseudocódigo Explicado

```
PSEUDOCODIGO  FILOSOFOS COMENSALES (Dijkstra con semaforos)

CONSTANTES:
    N
    THINKING <- 0
    HUNGRY <- 1
    EATING <- 2

FUNCIONES AUXILIARES:
    LEFT(i) <- (i + N - 1) mod N
    RIGHT(i) <- (i + 1) mod N

VARIABLES COMPARTIDAS:
    state[0..N-1]
    semaphore mutex
    semaphore s[0..N-1]

PROCEDIMIENTO test(i):
    if state[i] == HUNGRY and
       state[LEFT(i)] != EATING and
```

```
      state[RIGHT(i)] != EATING then

       state[i] <- EATING
       signal(s[i])
    end if

PROCEDIMIENTO take_forks(i):
    wait(mutex)
       state[i] <- HUNGRY
       test(i)
    signal(mutex)

    wait(s[i]) // se bloquea si no puede comer
    // aqui ya esta en EATING

PROCEDIMIENTO put_forks(i):
    wait(mutex)
       state[i] <- THINKING
       test(LEFT(i))
       test(RIGHT(i))
    signal(mutex)

RUTINA philosopher(i):
    repetir ROUNDS veces:
       pensar
       take_forks(i)
       comer
       put_forks(i)
```

1.7. Justificación de Corrección

Exclusión mutua de cubiertos: un filósofo solo entra en estado `EATING` si ambos vecinos **no** están comiendo. Por tanto, no hay uso simultáneo de un mismo cubierto.

No hay deadlock: la habilitación a comer se hace mediante `test(i)` bajo protección de `mutex`, evitando la espera circular: o queda habilitado completo, o queda bloqueado.

Concurrencia real: filósofos no vecinos pueden comer simultáneamente, pues sus condiciones no interfieren.

1.8. Implementación en C

Repositorio: GitHub: trabajo1.c

```c
#define _DEFAULT_SOURCE // expone usleep/useconds_t con -std=c11 -pedantic
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>
```

```
#ifndef N_PHILOS
#define N_PHILOS 5
#endif

#ifndef ROUNDS
#define ROUNDS 1
#endif

#ifndef THINK_US
#define THINK_US 700000
#endif

#ifndef EAT_US
#define EAT_US 700000
#endif

typedef enum { THINKING, HUNGRY, EATING } state_t;

static state_t state[N_PHILOS];
static sem_t mutex;
static sem_t s[N_PHILOS];

static pthread_t th[N_PHILOS];
static int ids[N_PHILOS];

static struct timespec t0;

static inline int LEFT(int i) { return (i + N_PHILOS - 1) % N_PHILOS; }
static inline int RIGHT(int i) { return (i + 1) % N_PHILOS; }

static long now_ms_since_start(void) {
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    long sec = (long)(t.tv_sec - t0.tv_sec);
    long nsec = (long)(t.tv_nsec - t0.tv_nsec);
    return sec * 1000L + nsec / 1000000L;
}

static void log_event(int i, int round, const char *msg) {
    printf("[%6ld ms] F[%d] R[%d/%d] %s\n",
           now_ms_since_start(), i + 1, round, ROUNDS, msg);
    fflush(stdout);
}

static void test(int i) {
    if (state[i] == HUNGRY &&
        state[LEFT(i)] != EATING &&
        state[RIGHT(i)] != EATING) {

        state[i] = EATING;
        sem_post(&s[i]);
```

```
    }
}

static void take_forks(int i, int round) {
    sem_wait(&mutex);

    state[i] = HUNGRY;
    log_event(i, round, "hambriento...");
    test(i);

    sem_post(&mutex);

    sem_wait(&s[i]);

    char buf[128];
    snprintf(buf, sizeof(buf), "usa cubiertos (%d, %d) y come...",
            i + 1, RIGHT(i) + 1);
    log_event(i, round, buf);
}

static void put_forks(int i, int round) {
    sem_wait(&mutex);

    state[i] = THINKING;

    char buf[128];
    snprintf(buf, sizeof(buf), "regresa los cubiertos (%d, %d)",
            i + 1, RIGHT(i) + 1);
    log_event(i, round, buf);

    test(LEFT(i));
    test(RIGHT(i));

    sem_post(&mutex);
}

static void *philosopher(void *arg) {
    int i = *(int *)arg;

    for (int r = 1; r <= ROUNDS; r++) {
        log_event(i, r, "piensa...");
        usleep((useconds_t)THINK_US);

        take_forks(i, r);
        usleep((useconds_t)EAT_US);

        put_forks(i, r);
    }

    printf("[%6ld ms] F[%d] FIN: completo %d rondas.\n",
          now_ms_since_start(), i + 1, ROUNDS);
```

```
    fflush(stdout);

    return NULL;
}

int main(void) {
    clock_gettime(CLOCK_MONOTONIC, &t0);

    printf("=== Filosofos comensales | N=%d | RONDAS=%d | THINK_US=%d | EAT_US
        =%d ===\n\n",
           N_PHILOS, ROUNDS, THINK_US, EAT_US);
    fflush(stdout);

    if (sem_init(&mutex, 0, 1) != 0) {
        perror("sem_init(mutex)");
        return 1;
    }

    for (int i = 0; i < N_PHILOS; i++) {
        state[i] = THINKING;
        if (sem_init(&s[i], 0, 0) != 0) {
            perror("sem_init(s[i])");
            return 1;
        }
    }

    for (int i = 0; i < N_PHILOS; i++) {
        ids[i] = i;
        if (pthread_create(&th[i], NULL, philosopher, &ids[i]) != 0) {
            perror("pthread_create");
            return 1;
        }
    }

    for (int i = 0; i < N_PHILOS; i++) pthread_join(th[i], NULL);

    for (int i = 0; i < N_PHILOS; i++) sem_destroy(&s[i]);
    sem_destroy(&mutex);

    printf("\n=== Listo: todos los filosofos completaron %d rondas. ===\n",
        ROUNDS);
    return 0;
}
```

1.9. Evidencia de Ejecución

Para la evidencia se configuró `ROUNDS = 1` con el fin de que la salida se visualice completa y legible en una sola captura.

```
~/Doc/College/2026/SistemasOperativos/T/trabajo1  main ?1  ./trabajo1
=== Filosofos comensales | N=5 | RONDAS=1 | THINK_US=700000 | EAT_US=700000 ===

[     0 ms] F[1] R[1/1] piensa...
[     0 ms] F[2] R[1/1] piensa...
[     0 ms] F[3] R[1/1] piensa...
[     0 ms] F[4] R[1/1] piensa...
[     0 ms] F[5] R[1/1] piensa...
[   701 ms] F[1] R[1/1] hambriento...
[   701 ms] F[1] R[1/1] usa cubiertos (1, 2) y come...
[   701 ms] F[2] R[1/1] hambriento...
[   701 ms] F[3] R[1/1] hambriento...
[   701 ms] F[3] R[1/1] usa cubiertos (3, 4) y come...
[   701 ms] F[4] R[1/1] hambriento...
[   701 ms] F[5] R[1/1] hambriento...
[  1401 ms] F[1] R[1/1] regresa los cubiertos (1, 2)
[  1401 ms] F[1] FIN: completo 1 rondas.
[  1401 ms] F[3] R[1/1] regresa los cubiertos (3, 4)
[  1401 ms] F[3] FIN: completo 1 rondas.
[  1401 ms] F[5] R[1/1] usa cubiertos (5, 1) y come...
[  1401 ms] F[2] R[1/1] usa cubiertos (2, 3) y come...
[  2102 ms] F[5] R[1/1] regresa los cubiertos (5, 1)
[  2102 ms] F[5] FIN: completo 1 rondas.
[  2102 ms] F[2] R[1/1] regresa los cubiertos (2, 3)
[  2102 ms] F[2] FIN: completo 1 rondas.
[  2102 ms] F[4] R[1/1] usa cubiertos (4, 5) y come...
[  2802 ms] F[4] R[1/1] regresa los cubiertos (4, 5)
[  2802 ms] F[4] FIN: completo 1 rondas.

=== Listo: todos los filosofos completaron 1 rondas. ===
~/Doc/College/2026/SistemasOperativos/T/trabajo1  main ?1
```

Figura 1: Salida de la terminal (una ronda).

1.10. Análisis de Resultados

Se observa que:

- Todos los filósofos completan la ejecución correctamente (en la evidencia: `1` ronda).
- No se presentan bloqueos permanentes (*deadlock*).
- La ejecución muestra intercalado de eventos, evidenciando concurrencia real.
- Los *timestamps* permiten verificar que no vecinos pueden comer en intervalos cercanos.

1.11. Diagramas

1.11.1. Diagrama de Estados del Filósofo

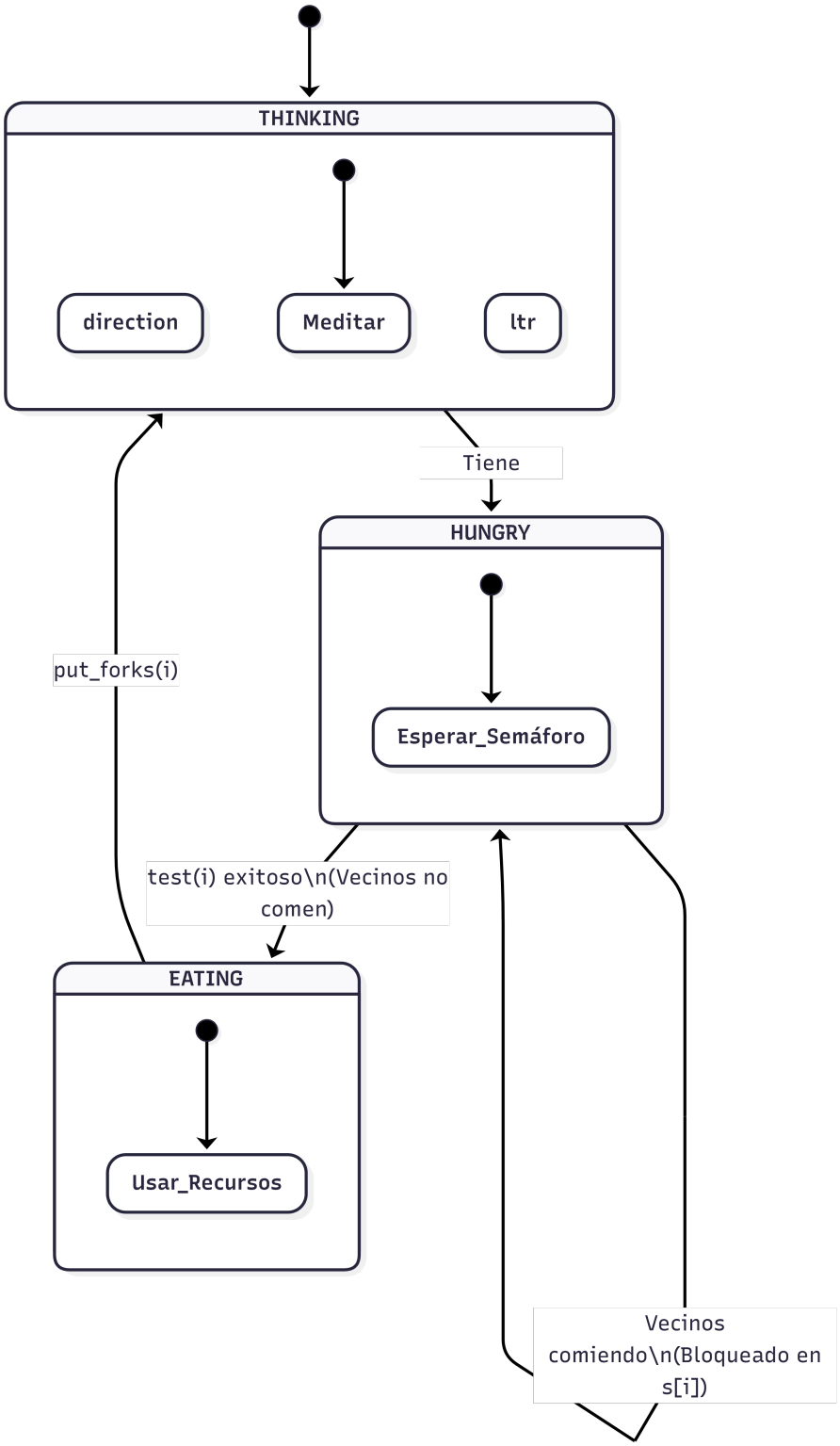

Figura 2: Diagrama de estados del filósofo.

1.11.2. Diagrama de Recursos y Semáforos

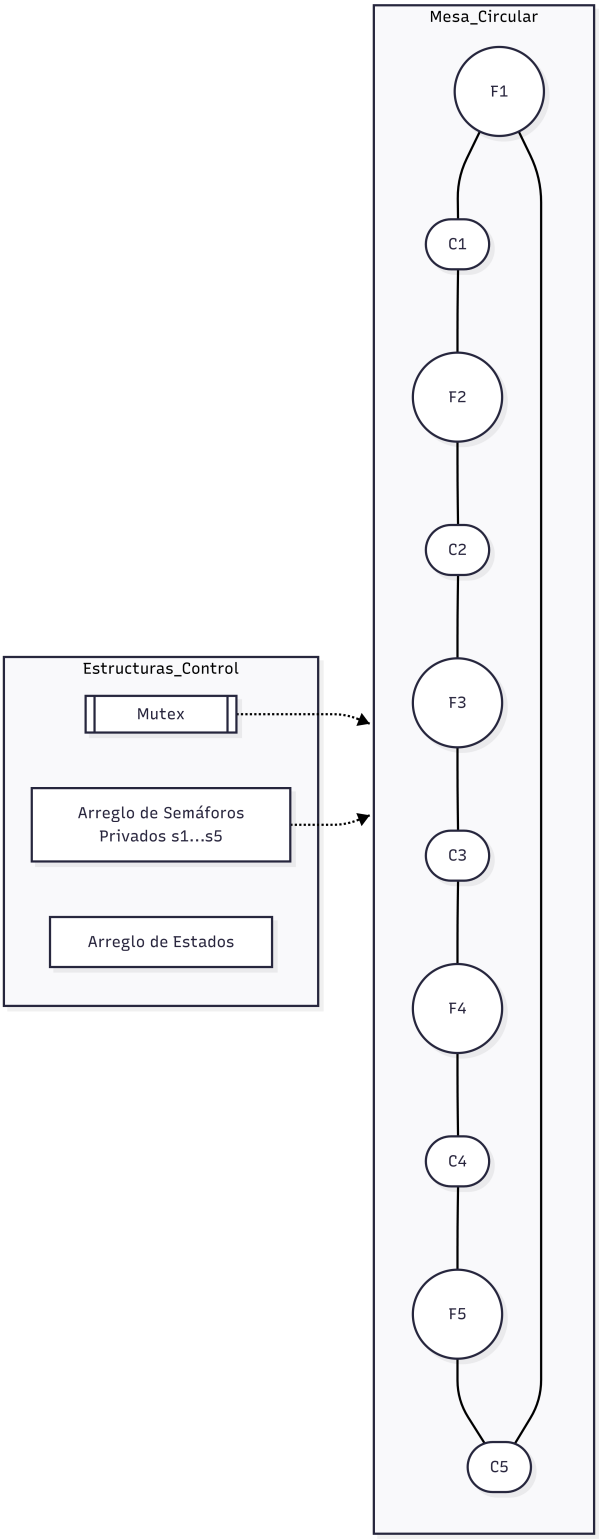

Figura 3: Recursos compartidos y estructuras de control (mutex y semáforos privados).

1.11.3. Diagrama de Sincronización (Lógica del Algoritmo)

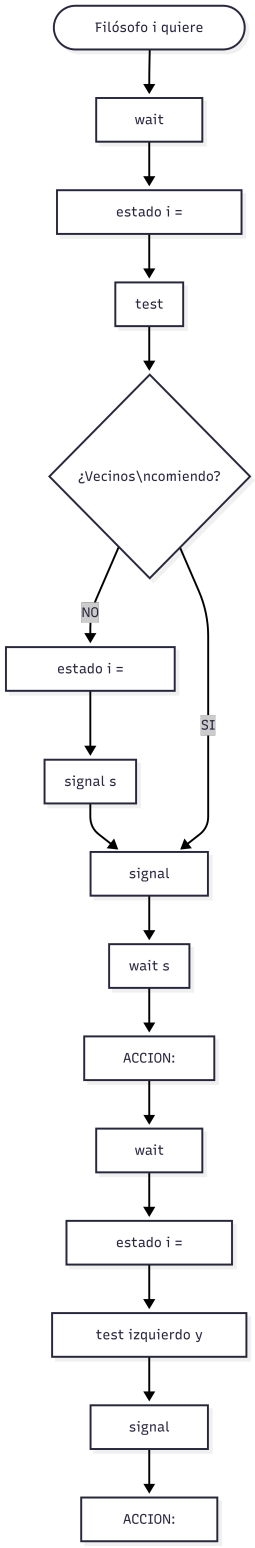

Figura 4: Flujo lógico de sincronización con `wait`, cambio de estado y `test`.

1.12. Conclusión

Se implementó correctamente el problema de los Filósofos Comensales utilizando semáforos POSIX. La solución evita interbloqueo, permite ejecución concurrente y sincroniza correctamente los accesos a recursos. Además, es configurable mediante macros para cambiar el número de filósofos y el número de rondas.